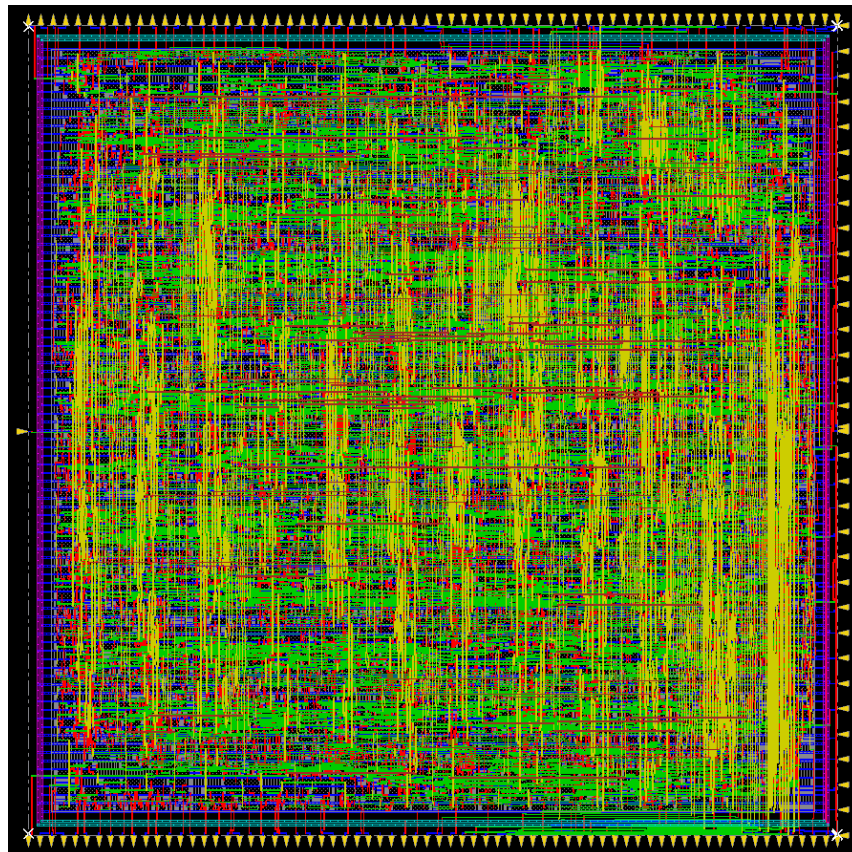




Appunti per tesi [Risc-V] (in corso)

Capitolo 1. Introduzione.



(Architettura RISK-V) fa PAURA, letteralmente.

Da sempre l'uomo ha la voglia di automatizzare, semplificare delle situazioni, lo fa dall'alba dei tempi. Il primo modello di "computer" ci arriva dai Maya che provarono a predire le eclissi con dei sistemi molto avanzati se contestualizzati al periodo...

Tornando ai giorni nostri (più o meno) negli anni '30 iniziarono a nascere i primi calcolatori per come li conosciamo noi erano a valvole e semimeccanici e potevano eseguire specifiche operazioni, non erano programmabili. Se ci pensate era un enorme spreco, come se oggi inventassero un pc che fa solo una cosa.

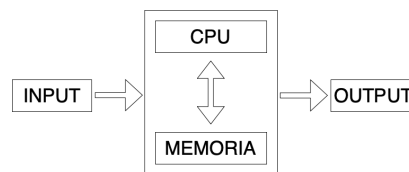
I primi pc programmabili arrivano a fine anni '40, prima programmabili con schede forate, poi programmabili in binario, MA CHE FATICA! Per questo iniziarono a nascere i linguaggi di programmazione dei linguaggi comodi agli umani e alle macchine in grado di semplificare il lavoro degli schiavi programmatori che poveretti per scrivere un ciclo for sulle schede forate non vedevano i figli per 9 giorni.

Compilatore

Un compilatore è un software che traduce quello che l'uomo scrive in linguaggio leggibile dall'uomo in codice binario leggibile dalla macchina.

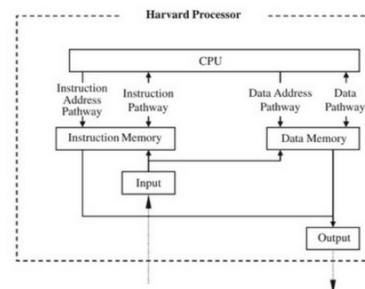
Macchina Teorica di Neumann

Neumann teorizzò che la macchina perfetta dovesse avere la seguente struttura:



tuttora i moderni calcolatori usano questa struttura di base.

Architettura Harvard



Un computer è diviso in:

- Memoria distinta per:
 - dati
 - programmi (istruzioni)
- CPU (CU + ALU + registri)
- Bus di comunicazione tra le diverse parti
Bus separato per le diverse memorie
- Periferiche di I/O
(tastiera, schermo, stampante, scheda di rete eccetera)

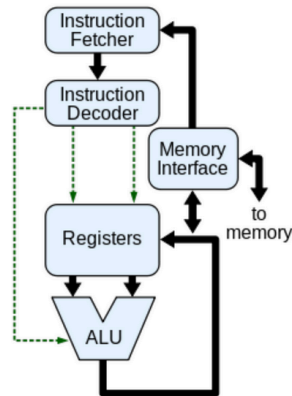
La CPU

La Central Processing Unit è formata da:

- La Unità di elaborazione Aritmetico/Logica (ALU)
fa solo calcoli

NON ha uno stato interno

- I registri
A uso generale e speciale,
per manipolare istruzioni, indirizzi, dati, risultati
- Il bus di comunicazione con la Memoria
- Il bus di comunicazione con le periferiche



Risc-V

Le architetture sono di due tipi principalmente, risk e cisk.

Risc = Reduced instruction set computer.

Cisc = Complex instruction set computer.

le principali differenze sono che:

nelle architetture risc le istruzioni hanno lunghezza fissa, le operazioni non attingono dati dalla memoria ma solo dai registri, c'è un modo di indirizzare la memoria semplice e di conseguenza la pipeline è più veloce.

Importante:

- Memoria RISC-V è indicizzata al byte
- Se sono all'indirizzo t e devo leggere la parola successiva incremento indirizzo come $t+4$, questo perché una parola sono 4 byte ossia 32 bit (1 byte = 8 bit)
- Il RISC-V non ha il vincolo di allineamento: le parole (DATI) non devono iniziare ad indirizzi multipli di 4 (ma per motivi di prestazione e atomicità delle istruzioni si preferisce allineare)
- Gli indirizzi sono little-endian (byte meno significativo è memorizzato per primo, nella locazione di memoria con indirizzo minore)

Comandi Risc-V

Linguaggio assembler RISC-V

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Aritmetiche	Somma	add x5, x6, x7	$x5 = x6 + x7$	Operandi in tre registri
	Sottrazione	sub x5, x6, x7	$x5 = x6 - x7$	Operandi in tre registri
	Somma immediata	addi x5, x6, 20	$x5 = x6 + 20$	Utilizzata per sommare delle costanti
Trasferimento dati	Lettura parola	lw x5, 40(x6)	$x5 = \text{Memoria}[x6 + 40]$	Spostamento di una parola da memoria a registro
	Lettura parola, senza segno	lwu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una parola senza segno da memoria a registro
	Memorizzazione parola	sw x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di una parola da registro a memoria
	Lettura mezza parola	lh x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una mezza parola da memoria a registro
	Lettura mezza parola, senza segno	lhu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una mezza parola senza segno da memoria a registro
	Memorizzazione mezza parola	sh x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di una mezza parola da registro a memoria
	Lettura byte	lb x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di un byte da memoria a registro
	Lettura byte, senza segno	lbu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di un byte senza segno da memoria a registro
	Memorizzazione byte	sb x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di un byte da registro a memoria
	Lettura di una parola e blocco	lr.d x5, (x6)	$x5 = \text{Memoria}[x6]$	Caricamento di una parola come prima fase di un'operazione atomica sulla memoria
	Memorizzazione condizionata di una parola	sc.d x7, x5, (x6)	$\text{Memoria}[x6] = x5; x7 = 0/1$	Memorizzazione di una parola come seconda fase di un'operazione atomica sulla memoria
	Caricamento costante nella mezza parola superiore	lui x5, 0x12345	$x5 = 0x12345000$	Caricamento di una costante su 20 bit nei 12 bit più significativi di una parola
Logiche	And	and x5, x6, x7	$x5 = x6 \& x7$	Operandi in tre registri; AND bit a bit
	Or inclusivo	or x5, x6, x8	$x5 = x6 x8$	Operandi in tre registri; OR bit a bit
	Or esclusivo (Xor)	xor x5, x6, x9	$x5 = x6 \wedge x9$	Operandi in tre registri; XOR bit a bit
	And immediato	andi x5, x6, 20	$x5 = x6 \& 20$	AND bit a bit tra un operando in registro e una costante
	Or esclusivo immediato	ori x5, x6, 20	$x5 = x6 20$	OR bit a bit tra un operando in registro e una costante
	Xor immediato	xori x5, x6, 20	$x5 = x6 \wedge 20$	XOR bit a bit tra un operando in registro e una costante
Scorrimento (shift)	Scorrimento logico a sinistra	sll x5, x6, x7	$x5 = x6 \ll x7$	Scorrimento a sinistra mediante registro
	Scorrimento logico a destra	srl x5, x6, x7	$x5 = x6 \gg x7$	Scorrimento a destra mediante registro
	Scorrimento aritmetico a destra	sra x5, x6, x7	$x5 = x6 \gg x7$	Scorrimento a destra aritmetico mediante registro
	Scorrimento logico a sinistra immediato	slli x5, x6, 3	$x5 = x6 \ll 3$	Scorrimento a sinistra mediante costante
	Scorrimento logico a destra immediato	srl i x5, x6, 3	$x5 = x6 \gg 3$	Scorrimento a destra mediante costante
	Scorrimento aritmetico a destra immediato	srai x5, x6, 3	$x5 = x6 \gg 3$	Scorrimento a destra aritmetico mediante costante
Salti condizionati	Salta se uguale	beq x5, x6, 100	Se $(x5 == x6)$ vai a PC+100	Test di uguaglianza; salto relativo al PC
	Salta se non è uguale	bne x5, x6, 100	Se $(x5 != x6)$ vai a PC+100	Test di disuguaglianza; salto relativo al PC
	Salta se minore di	blt x5, x6, 100	Se $(x5 < x6)$ vai a PC+100	Comparazione di minoranza; salto relativo al PC
	Salta se maggiore o uguale di	bge x5, x6, 100	Se $(x5 \geq x6)$ vai a PC+100	Comparazione di maggioranza o uguaglianza; salto relativo al PC
	Salta se minore di, senza segno	bltu x5, x6, 100	Se $(x5 < x6)$ vai a PC+100	Comparazione di minoranza senza segno; salto relativo al PC
	Salta se maggiore o uguale di, senza segno	bgeu x5, x6, 100	Se $(x5 \geq x6)$ vai a PC+100	Comparazione di maggioranza o uguaglianza senza segno; salto relativo al PC
Salti incondizionati	Salta e collega	jal x1, 100	$x1 = PC+4$; vai a PC+100	Chiamata a procedura con indirizzamento relativo al PC
	Salta e collega mediante registro	jalr x1, 100(x5)	$x1 = PC+4$; vai a $x5+100$	Ritorno da procedura; chiamata indiretta

Formato delle istruzioni RISC-V

Istruzioni della CPU tutte da 32 bit con formato molto simile

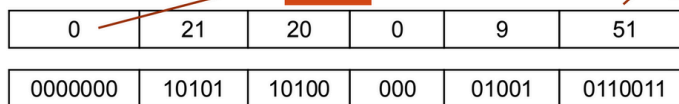
R-type: (tipo a Registro)

- SENZA accesso alla memoria
- Istruzioni Aritmetico/Logiche



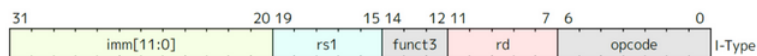
Esempio

add x9, x20, x21



I-type: (tipo Immediato)

- Load
- Somma immediata



Esempio: se voglio sommare al valore in un registro un intero...

S-type: (tipo Store)

- Salvataggio registro in memoria

